

Programación Web

TEMA 6. SERVICIOS

WEB

Curso 2025-2026

Actualización: 14/5/2026

Bibliografía

- E. Cerami, «[Web Services Essentials](#)», O'Reilly, 2002
- D. Chappell y T. Jewell, «Java Web Services», O'Reilly, 2002
- L.J. Mitchel, “PHP Web Services”, O'Reilly, 2013.
- E. Newcomer, «[Understanding Web Services](#)», Addison-Wesley Professional, 2002
- W3C, «Web Services Architecture»,
<http://www.w3.org/TR/ws-arch>
- [Introducción a los servicios web \(UA\)](#)

Contenido

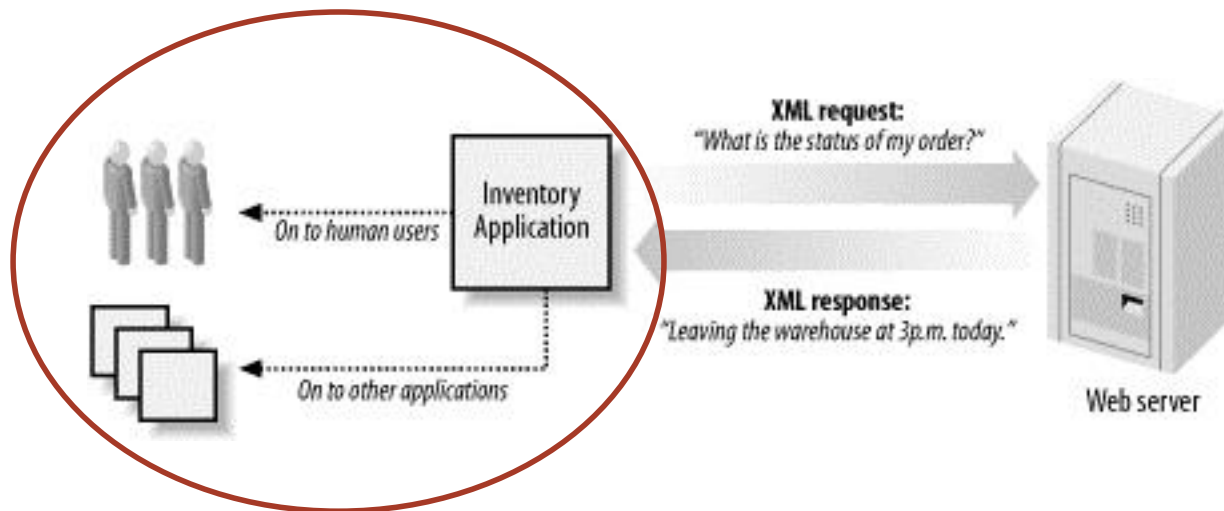
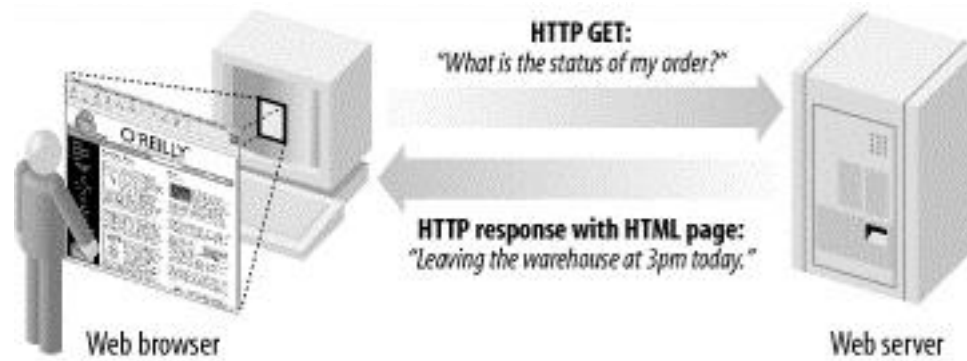
Definición

Conceptos básicos

Arquitectura

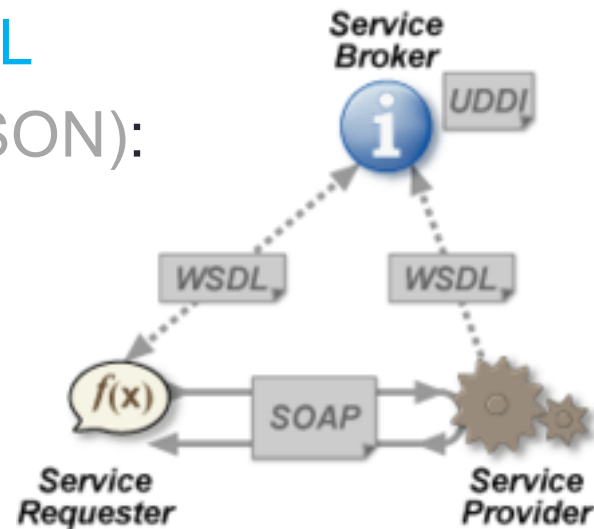
Ejemplos de código
disponibles en:
[http://bahia.ugr.es/~jmbs/
WebServices](http://bahia.ugr.es/~jmbs/WebServices)

DEFINICIONES



Servicio web

- Tecnología que utiliza un conjunto de protocolos y estándares para **intercambio de datos entre aplicaciones**.
- Cualquier **servicio** ofrecido a través de Internet, mediante de un sistema de **mensajes XML**
- Sistemas de mensajes XML (JSON):
 - XML-RPC
 - SOAP
 - HTTP GET/POST



Definición del W3C

Un **servicio web** es un sistema software diseñado para soportar la interacción **máquina-a-máquina**, a través de una red, de forma interoperable.

Interfaz descrita en un formato procesable por un equipo informático (**WSDL**).

Interacción mediante el intercambio de mensajes **SOAP**, típicamente transmitidos usando *serialización XML* sobre HTTP conjuntamente con otros estándares web.

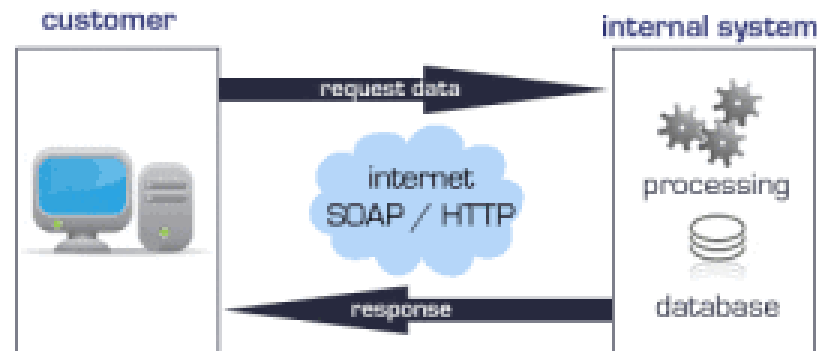


Eric Newcomer's definition



Web services provide a ***layer of abstraction*** above existing software systems, such as application servers, CORBA, .NET servers, messaging, and packaged applications. Web services work at a level of abstraction similar to the Internet and ***are capable of bridging*** any operating system, hardware platform, or programming language, just as the Web is.

Web services are Extensible Markup Language (XML) **applications mapped** to programs, objects, or databases or to comprehensive business functions.



A program sends a request to a Web service (message as an XML document) across the network, and, optionally, receives a reply, also in the form of an XML document.

Interfaces

[→ IBM Web Services Guide](#)

Recibe un mensaje en XML de la red



Transforma los datos XML en un formato procesable por el back-end



(Opcional) Devuelve un mensaje

Casos de uso



Verificación de tarjetas de crédito



Robots de compra



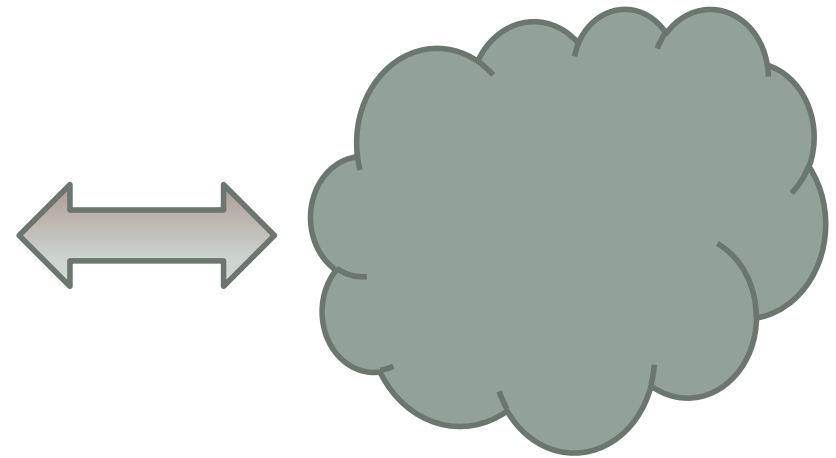
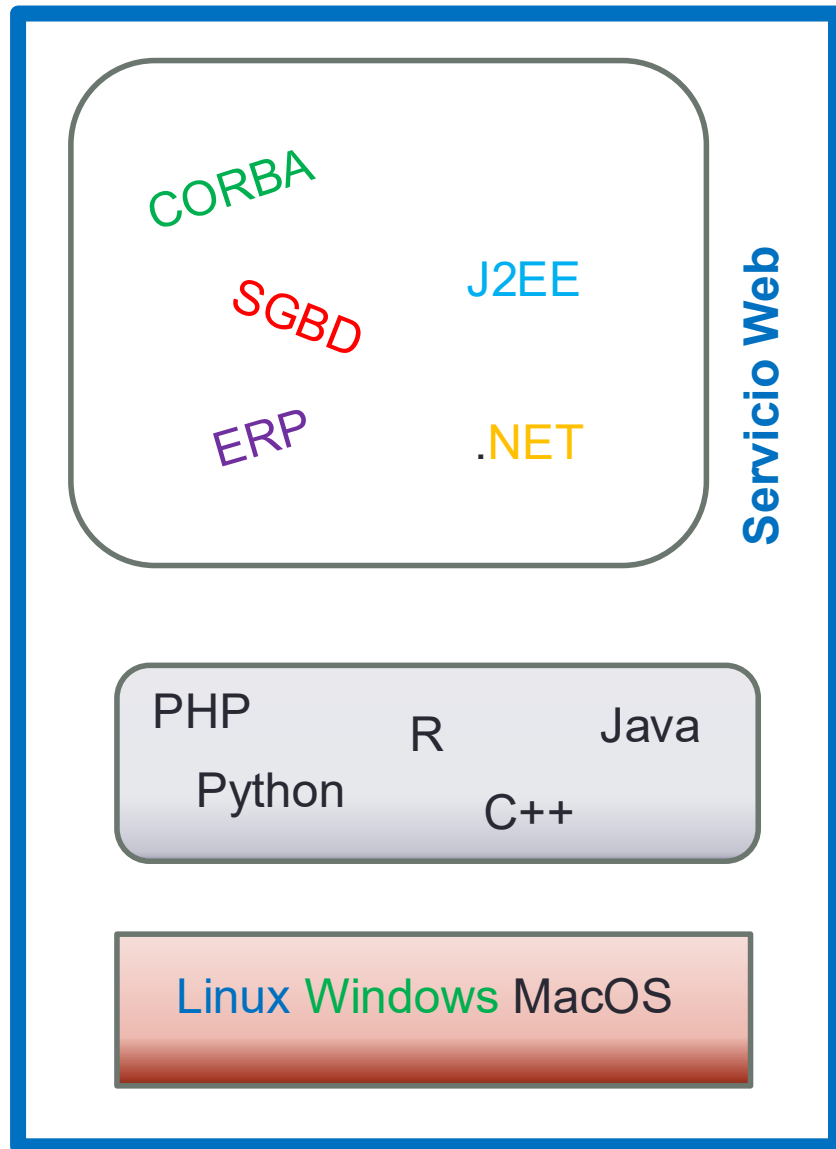
Conversión de divisas



Traducción de textos

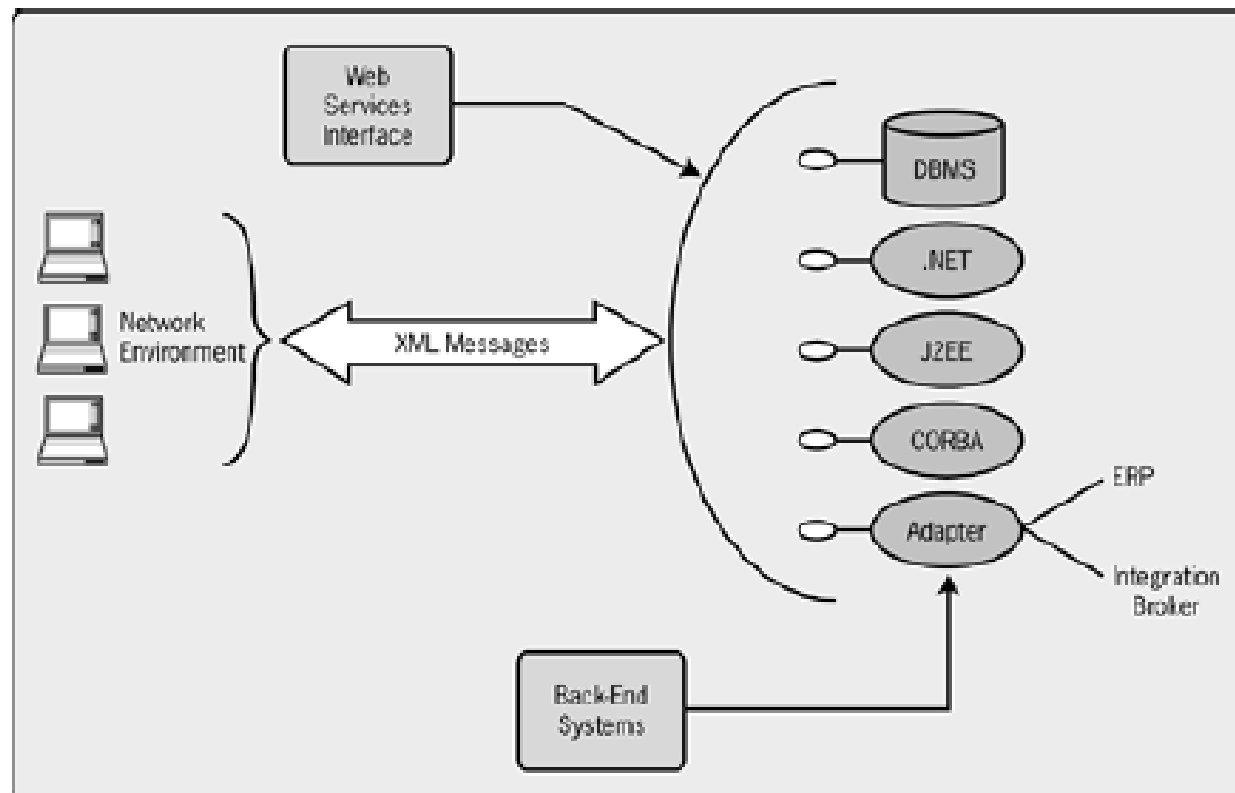


Gestión de carteras



Construcción de
aplicaciones web

Esquema



Propiedades

- Son componentes de aplicaciones
- Se comunican a través de protocolos abiertos
- Son **autocontenidos** y **autodescriptivos**
- **Descubribles** a través de UDDI
- Utilizables por otras aplicaciones

Ejemplo: búsqueda de información

Mecanismo habitual: insertar datos en URL:

<http://www.google.com/search?q=Skate+boots&btnG=Google+Search>

Búsqueda de “Skate boots” en el motor de búsqueda de Google:

- search: servicio requerido
- “Skate+boots”: cadena buscada, enviada en HTML

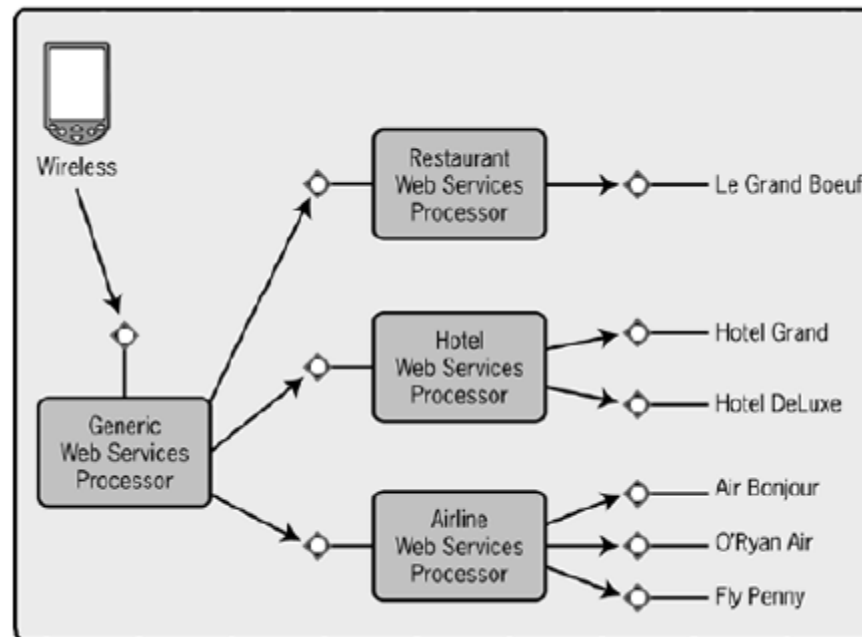
Alternativa XML

```
<SOAP-ENV:Body>  
  <s:SearchRequest  
    xmlns:s="www.xmlbus.com/SearchService">  
    <p1>Skate</p1>  
    <p2>boots</p2>  
  </s:SearchRequest>  
</SOAP-ENV:Body>
```


Ejemplos de aplicaciones

- Buscar y comprar bienes y servicios al mejor precio
 - Comprar billetes de avión; seguros de coche; hoteles, ...
- Coordinar billetes de viajes y eventos
- Gestión de procesos de negocio: consecución, facturación y envío
- ...

Coordinación de vuelo, alojamiento y restaurante



CONCEPTOS BÁSICOS

Agentes y servicios

El **agente** es el componente software que gestiona la comunicación (mensajes).

El **servicio** es el recurso, caracterizado por la funcionalidad que se provee

Ej.: el agente se puede implementar con distintos lenguajes de programación, mientras que el servicio es el mismo

Clientes y proveedores

- El **proveedor** es la entidad que ofrece un agente que implementa un servicio particular
- El **cliente** es la entidad que desea utilizar el servicio de un proveedor. Implementa un agente que dialoga con el agente del proveedor



Pila de protocolos

Discovery	UDDI
Description	WSDL
XML messaging	XML-RPC, SOAP, XML
Transport	HTTP, SMTP, FTP, BEEP

Interacción para un servicio web

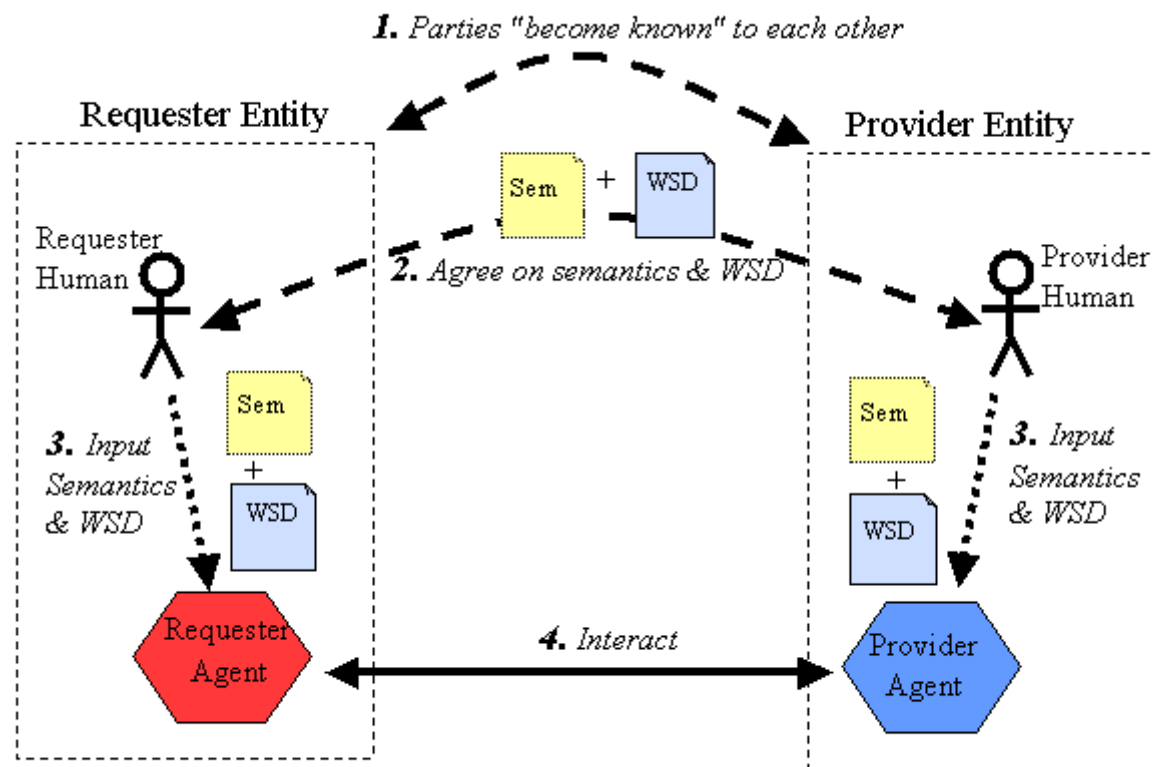


Figure 1-1. The General Process of Engaging a Web Service

ARQUITECTURA

Habilidades para interacción de programas a través de la web



Localizarse y descubrir información que les permita interaccionar

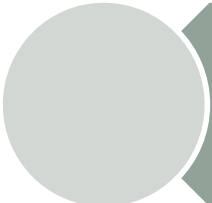


Averiguar los patrones de interacción: solicitud-respuesta o más elaborado



Negociar condiciones: calidad del servicio, fiabilidad en la comunicación, transacciones

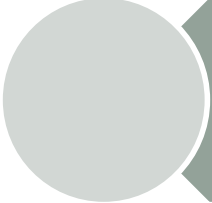
Estándares relacionados



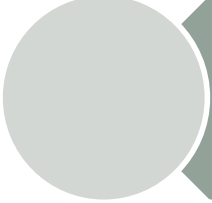
UDDI



WSDL



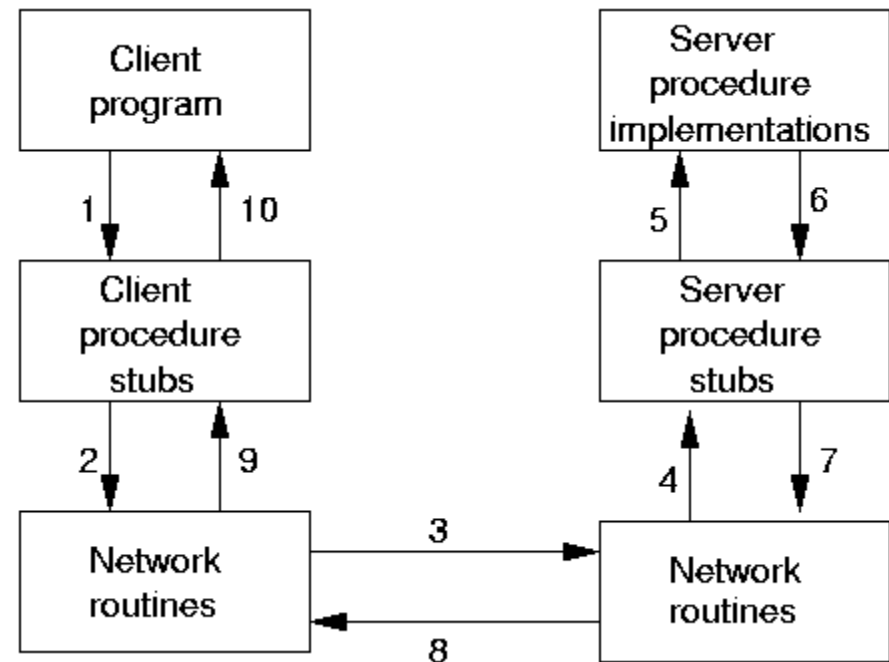
XML-RPC; SOAP



HTTP

Remote Procedure Call (RPC)

- RPC implementa un proceso de llamada a un procedimiento, pero en una ubicación remota (no local)
- Existe una complejidad adicional en el proceso de comunicación entre el invocador y el llamado
- Protocolos existentes para computación distribuida



XML en servicios web

- XML se creó para superar las limitaciones de HTML. Especialmente, para proporcionar mejor soporte en la **creación** y **gestión** de **contenido dinámico**
- Con XML puedes crear cualquier elemento que **asocie significado a los datos**: mucha flexibilidad
- Los esquemas restringen dicha flexibilidad

XML (Extensible Markup Language)

[→Tutorial XML de W3Schools](#)

- Se diseñó para almacenar e intercambiar datos
(Centrado en los datos)
- Es legible para seres humanos y máquinas
- Etiquetas no predefinidas
Esquemas
- Formato texto

Ventajas del envío en XML

- Mejor control de los tipos de datos y la estructura de la información
- Más flexibilidad
- Más extensible
- ...

Desventajas

- Para transacciones, el desarrollo es mucho menos maduro que el de estándares de computación distribuida como CORBA
- El rendimiento es muy bajo, comparado con otros modelos de computación distribuida. RMI, CORBA
Entre los objetivos de XML no están la concisión, ni la eficiencia en el procesamiento

Justamente por su complejidad, aparecen alternativas más simples para el intercambio de datos: JSON, YAML

XML-RPC

Extensible Markup Language-Remote Procedure Call

- Protocolo que usa XML para ejecutar RPC
- Solicitudes codificadas en XML y enviadas vía POST
- Las respuestas XML se incrustan en el cuerpo de la respuesta HTTP
- Es independiente de plataforma

Similar a las cabeceras de funciones

Ejemplo: weather service

getWeather-request.xml

```
<methodCall>
  <methodName>weather.getWeather </methodName>
  <params>
    <param><value>10016</value></param>
  </params>
</methodCall>
```

getWeather-reply.xml

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<methodResponse>
  <params>
    <param>
      <value><int>65</int></value>
    </param>
  </params>
</methodResponse>
```

SOAP

Simple Object Access Protocol

- Protocolo de comunicaciones que define un formato de *serialización* para la transmisión de documentos XML sobre una red y para representar interacciones RPC
- Es más complejo que XML-RPC.
- Usa espacios de nombres y esquemas de XML.

Ejemplo: petición

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://www.w3.org/2001/09/soap-
envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <SOAP-ENV:Body>
    <ns1:getWeather
      xmlns:ns1="urn:examples:weatherservice"
      SOAP-ENV:encodingStyle=
        "http://www.w3.org/2001/09/soap-encoding/">
      <zipcode xsi:type="xsd:string">10016</zipcode>
    </ns1:getWeather>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

getWeather-SOAP-request.xml

Ejemplo: respuesta

```
<?xml version='1.0' encoding='Utf-8' >
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV=
    "http://www.w3.org/2001/09/soap-envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <SOAP-ENV:Body>
    <ns1:getWeatherResponse
      xmlns:ns1="urn:examples:weatherservice"
      SOAP-ENV:encodingStyle=
        "http://www.w3.org/2001/09/soap-encoding/">
      <return xsi:type="xsd:int">65</return>
    </ns1:getWeatherResponse>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

getWeather-SOAP-reply.xml

Descripción de los servicios

- El protocolo de intercambio de mensajes para prestación de servicios web se describe en el lenguaje **WSDL**
- La descripción del protocolo (WSD) incluye formatos de mensajes, tipos de datos, protocolos de transporte y *serialización*
- También indicaciones en las que localizar el servicio

WSDL

Web Services Description Language

Lenguaje, basado en XML, para definir

- Tipos de datos incluidos en los mensajes
- Operaciones a realizar sobre los mensajes
- La traducción de mensajes en redes de transporte (interconexión entre servicios)

Ejemplo: servicio weather (I)

WeatherService.wsdl

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="WeatherService"
  targetNamespace="http://www.ecerami.com/wsdl/WeatherService.wsdl"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://www.ecerami.com/wsdl/WeatherService.wsdl"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <message name="getWeatherRequest">
    <part name="zipcode" type="xsd:string"/>
  </message>
```

Servicio weather (II)

```
<message name="getWeatherResponse">
  <part name="temperature" type="xsd:int"/>
</message>

<portType name="Weather_PortType">
  <operation name="getWeather">
    <input message="tns:getWeatherRequest"/>
    <output message="tns:getWeatherResponse"/>
  </operation>
</portType>
```


Servicio weather (III)

```
<service name="Weather_Service">  
  <documentation>WSDL File for Weather  
  Service</documentation>  
  <port binding="tns:Weather_Binding"  
  name="Weather_Port">  
    <soap:address  
    location="http://localhost:8080/soap/servlet/rpcrouter"/>  
  </port>  
</service>  
</definitions>
```

WSDL en acción

- Con WSDL un cliente puede localizar un servicio web e invocar funciones disponibles automáticamente.
- **Automatizable** con herramientas adecuadas

```
java clients.DynamicInvoker  
http://localhost:8080/wsdl/WeatherService.wsdl getWeather  
10016
```

```
Reading WSDL document from  
'http://localhost:8080/wsdl/WeatherService.wsdl'  
Preparing WSIF dynamic invocation  
Executing operation getWeather  
Result:  
temperature=65  
Done!
```

UDDI

Registro global de servicios web

Universal Description, Discover, and Integration

Mecanismos de registro y descubrimiento de servicios web utilizados para almacenar y categorizar información de negocio y para recuperar direcciones de acceso a servicios web

Directorio distribuido de negocios y servicios web.
API para buscar datos y publicar nuevos datos

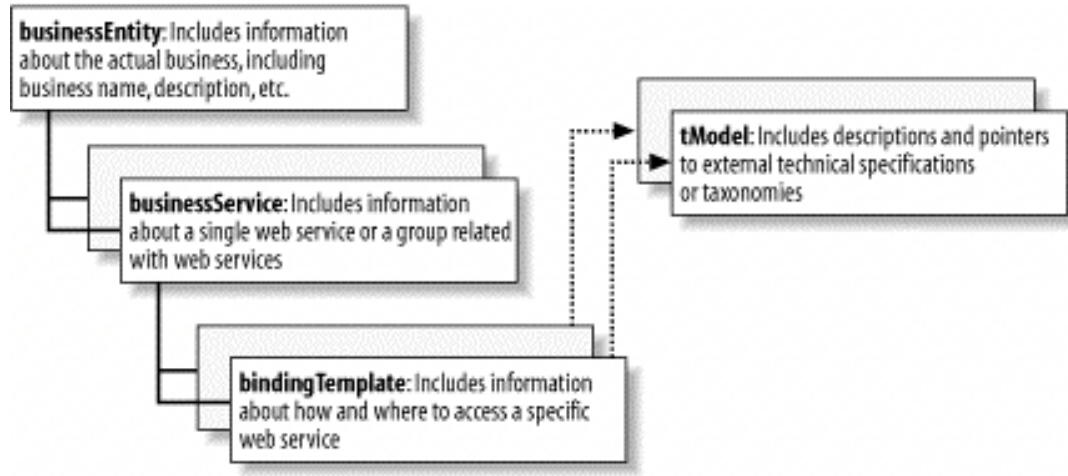
Implementación de la especificación UDDI

Arquitectura de UDDI

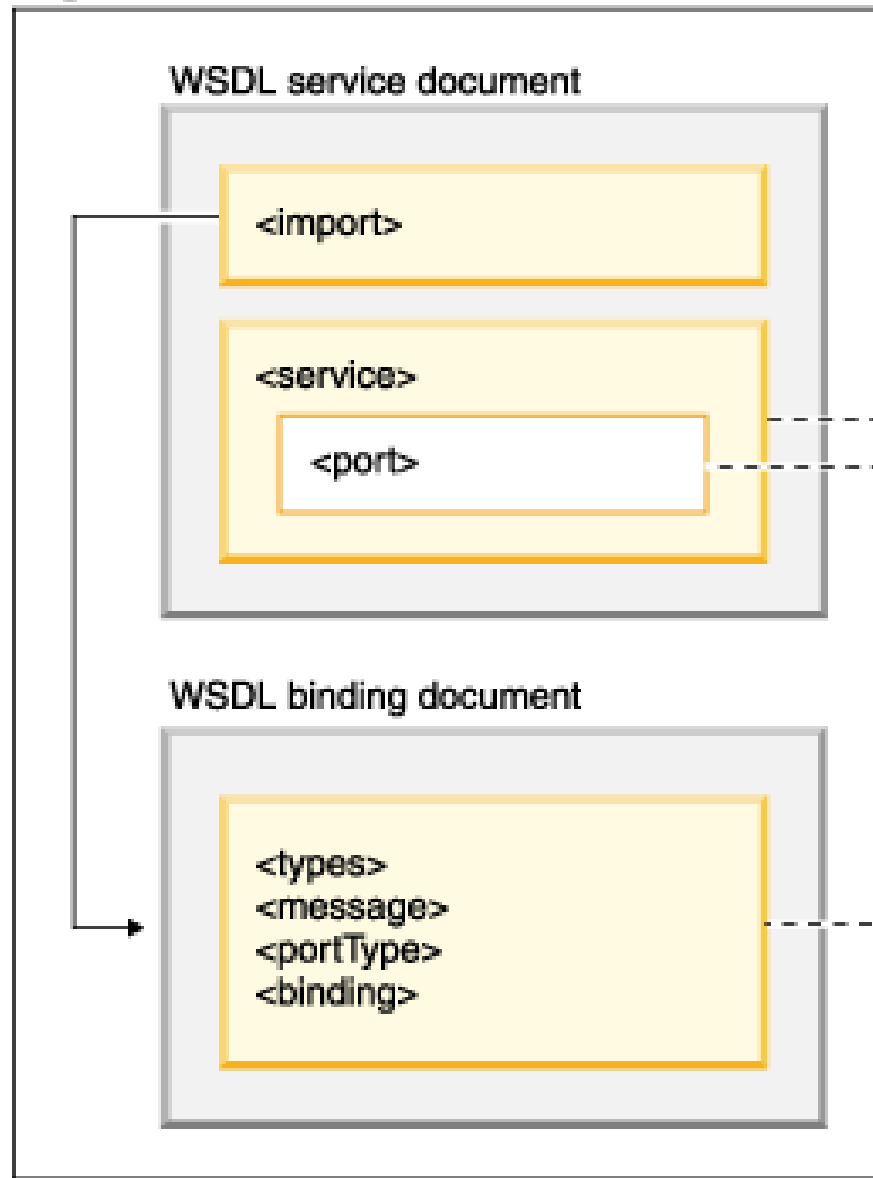
Modelo de datos:
Esquema XML para
descripción de servicios.

API: API basada en
SOAP para búsqueda y
publicación de datos.

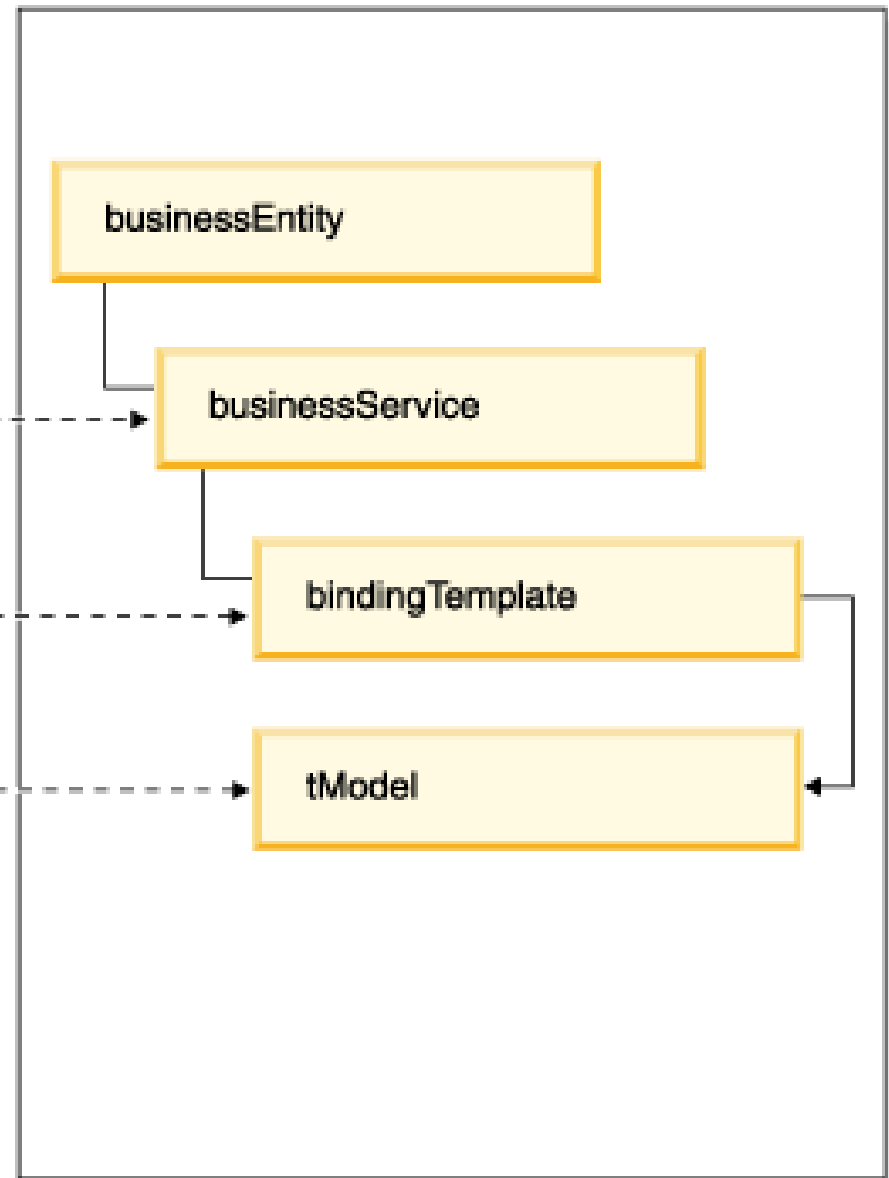
Servicios cloud:
Proveedores que aportan
implementaciones de
UDDI y sincronizan los
datos

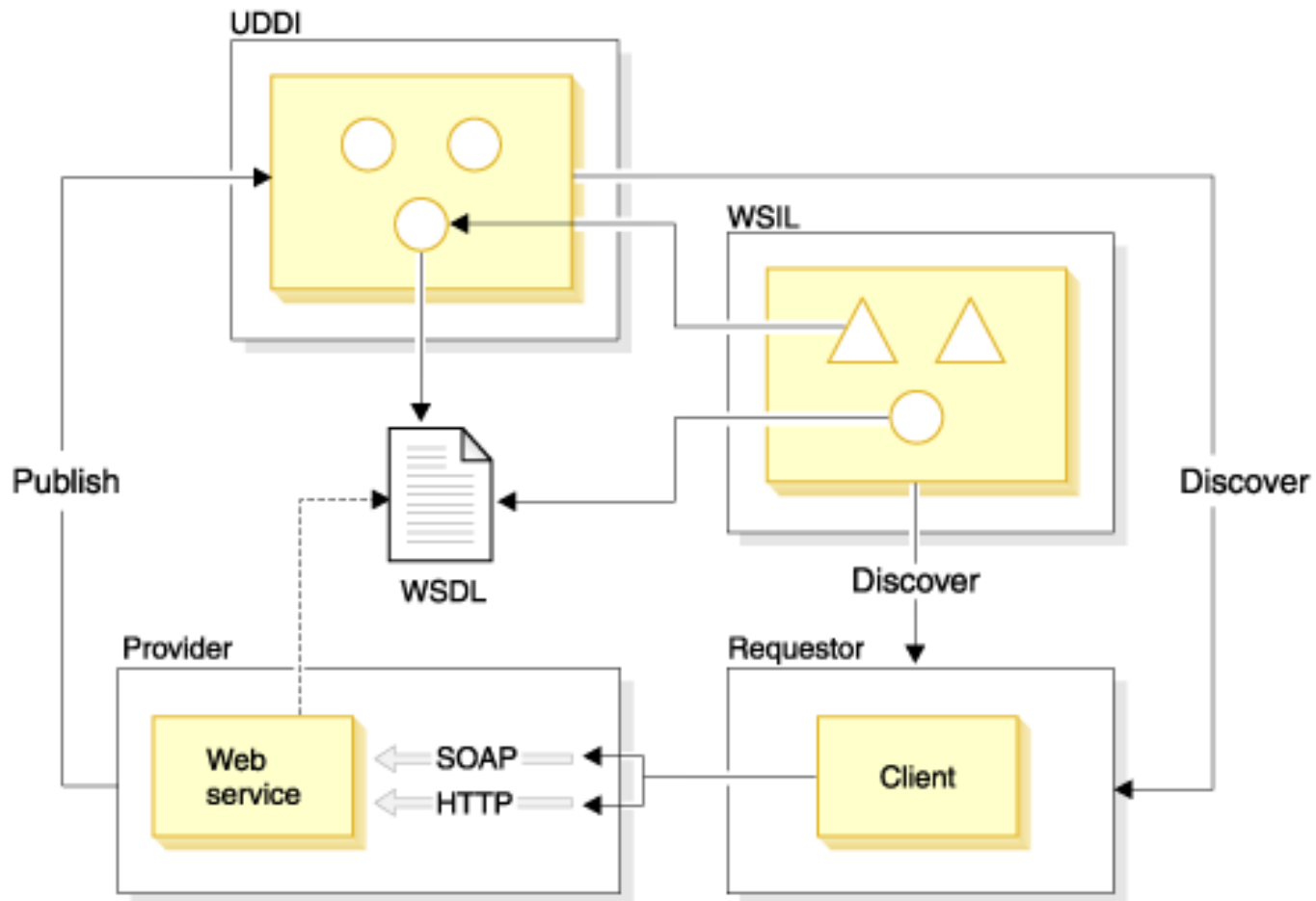


WSDL



UDDI

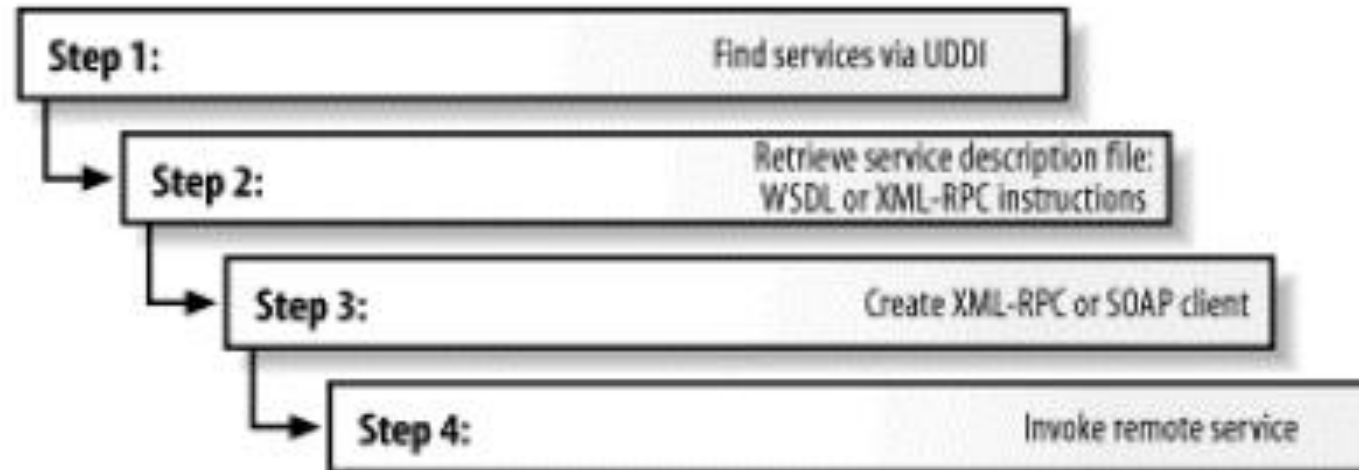




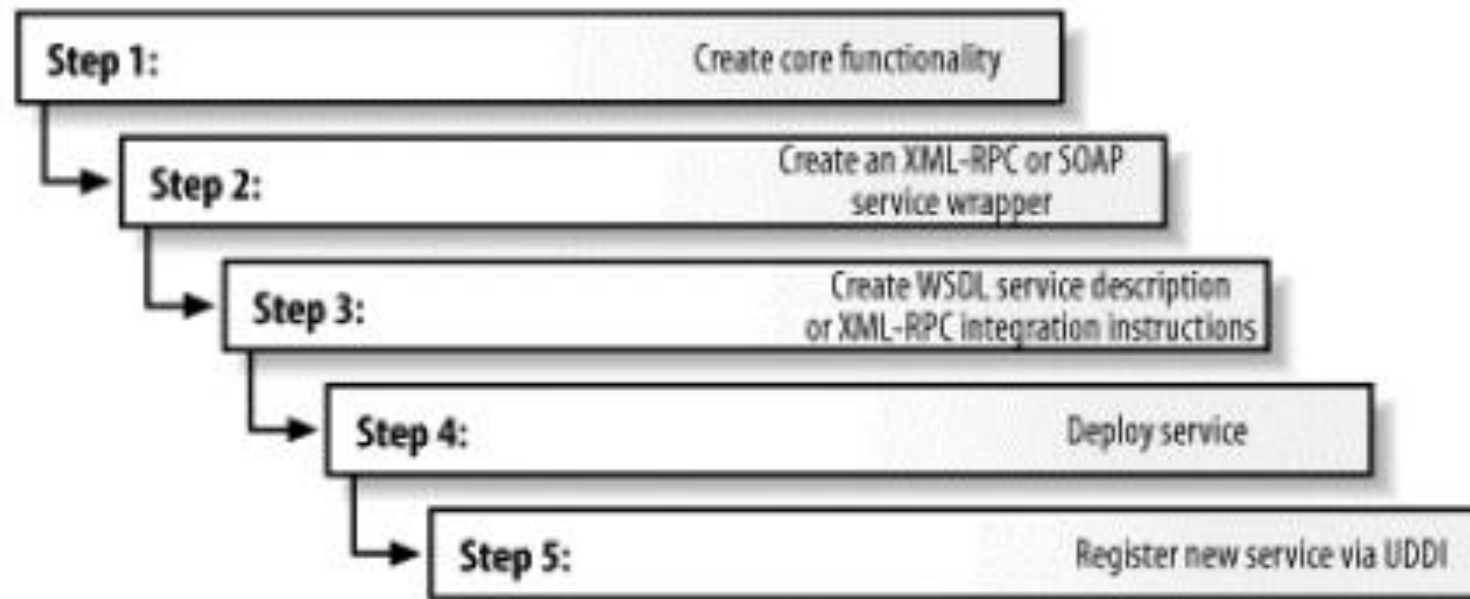
Legend

-  References to service descriptors
-  Pointers to WSDL documents
-  Originates from

Perspectiva del cliente



Perspectiva del proveedor



Beneficios de los servicios web

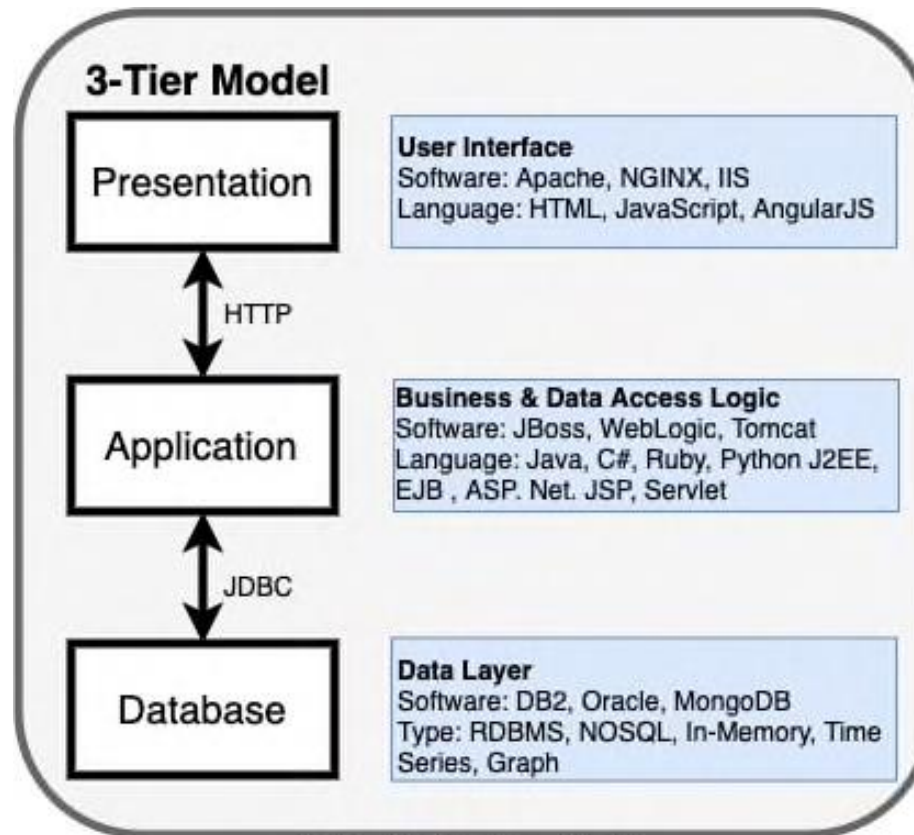
- Exponer la funcionalidad a la red
- Conectar distintas aplicaciones (interoperatividad)
- Protocolo estandarizado
- Bajo coste de comunicación

ARQUITECTURAS PARA APLICACIONES WEB

Algunos tipos de arquitecturas

- Arquitectura en tres capas
- Service Oriented Architecture (SOA)
 - REST
- Microservicios
- Serverless:
 - Function as a Service (FaaS)
- Arquitectura basada en la nube

Arquitectura en 3 capas

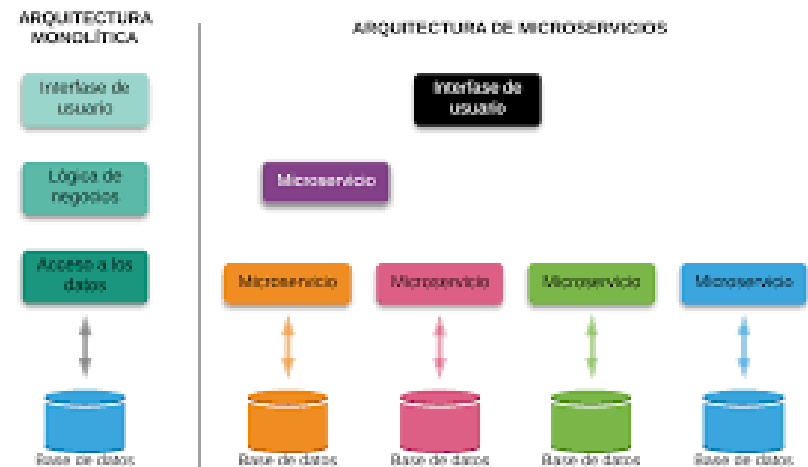


3-Tier Architecture Model

Microservicios

Estilo de desarrollo de aplicaciones software a partir de la composición de un conjunto de servicios pequeños, cada uno ejecutándose de forma autónoma y comunicándose entre sí.

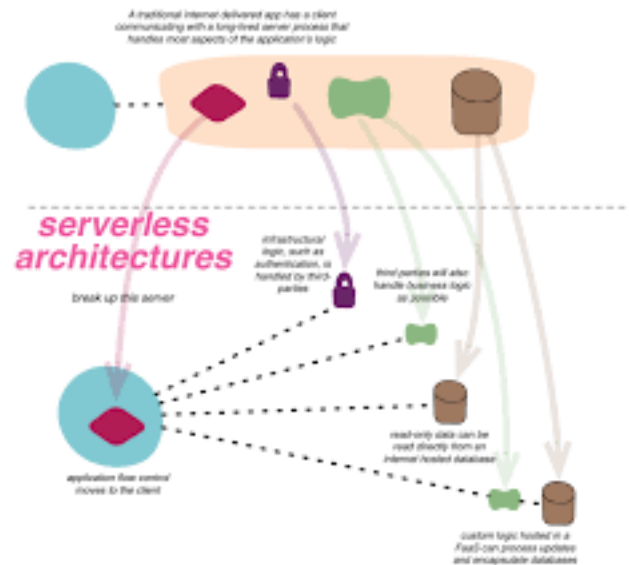
- Despliegue automatizado
- Control descentralizado de módulos y datos

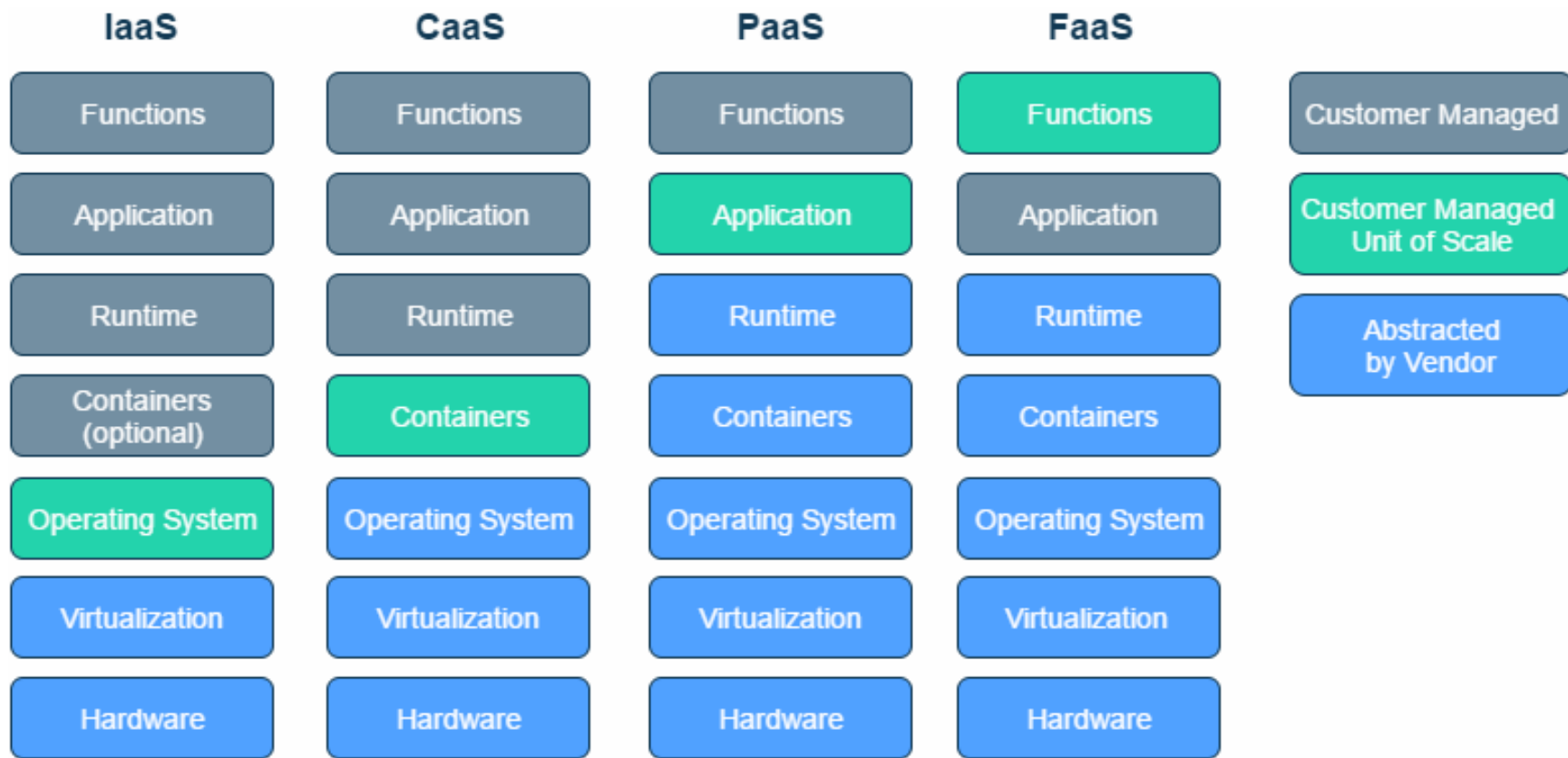


Serverless

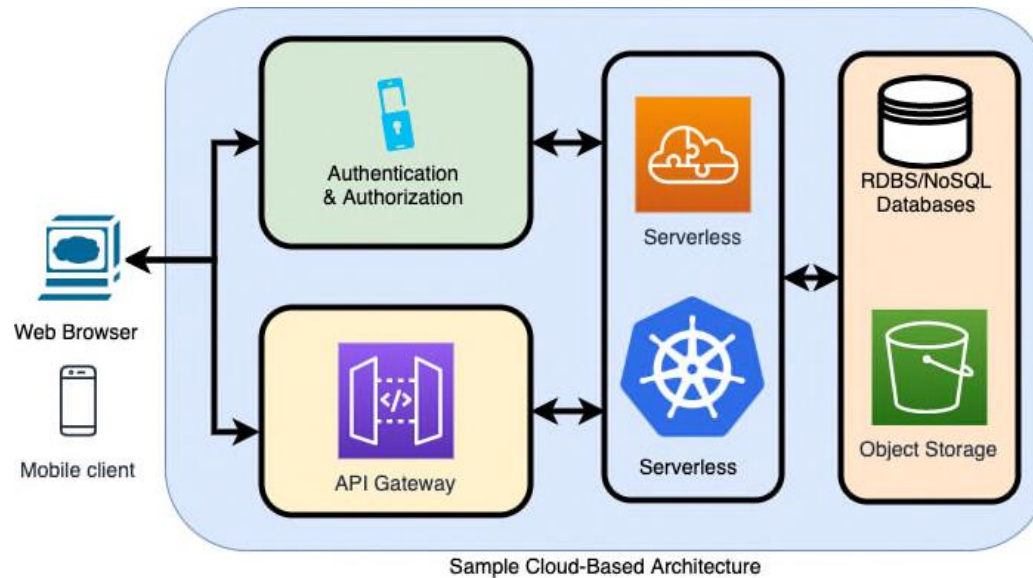
Modelo de computación

El proveedor de IaaS **permite ejecutar durante un periodo de tiempo determinado porciones de código denominadas "funciones"** sin necesidad de hacernos cargo de la infraestructura subyacente que se provisiona para dar el servicio





Arquitectura basada en la nube



ACTIVIDADES COMPLEMENTARIAS

Para profundizar

1. Estudiar con más profundidad XML:
 1. E.T. Ray, “Learning XML”, O’Reilly,
 2. E.R. Harold, W.S. Means, “XML in a Nutshell: A Desktop Quick Reference”, O’Reilly,
 3. [Tutorial XML de W3Schools](#)
2. Estudiar servicios *serverless* y FaaS de proveedores cloud:
 1. Amazon Lambda
 2. Azure Functions
 3. Google Cloud Functions
 4. OpenFaaS

Sobre desarrollo de servicios web

- [PHP RESTful Web Service API](#)
- [Rocket: Web service development in Rust](#)